

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 4 JDBC — Conexión a bases desde Java

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Alta y listado de clientes	2
3. Ejercicio 2: Invocar la función de máxima cotización	5
4. Ejercicio 3: Listado de metadatos (PK, UQ, FK)	8
5. Decisiones transversales	14
Conclusión	14

1. Introducción

El Práctico 4 cierra el bloque de fundamentos del cuatrimestre moviéndose desde SQL puro hacia el *cliente*: cómo se accede a las bases que armamos en prácticos anteriores desde una aplicación Java mediante la API **JDBC** (*Java DataBase Connectivity*). Los tres ejercicios cubren los tres usos canónicos:

- **Ejercicio 1:** `PreparedStatement` para alta y `ResultSet` para listado, sobre la base de facturación del Práctico 1.a (MySQL y PostgreSQL).
- **Ejercicio 2:** `CallableStatement` para invocar la función `fn_actualizar_cotizacion_maxima` creada en el Práctico 3, sobre PostgreSQL (y el procedimiento equivalente en MySQL).
- **Ejercicio 3:** `DatabaseMetaData` para listar tablas, columnas, claves primarias, únicas y foráneas de la base del Práctico 1.c, en PostgreSQL y Oracle (más MySQL como caso adicional).

Cada ejercicio sigue siempre la misma estructura: **Consigna** (texto literal del PDF), **Idea** (lectura natural y concepto JDBC detrás), **Solución** (código Java con su `.properties`), **Por qué funciona** (lógica paso a paso), **En MySQL 8+** (snippet equivalente con driver y URL de MySQL) y **Prueba** (compilación e invocación). Toda la solución usa exclusivamente `java.sql` (sin Hibernate ni JPA): JDBC pelado, como pide la consigna.

Convenciones de proyecto. Cada ejercicio vive en su carpeta (`ejercicio-01`, `ejercicio-02`, `ejercicio-03`) con la misma estructura: `src/` (código), `lib/` (drivers JDBC), `config/` (`*.properties`) y un `README.md` con instrucciones. Los datos sensibles (host, usuario, clave) viven en los `.properties`, no en el código. Todos los recursos JDBC se cierran con `try-with-resources`; todas las sentencias usan `?` y `PreparedStatement` (sin concatenación de strings).

2. Ejercicio 1: Alta y listado de clientes

Consigna

Teniendo en cuenta el ejercicio 1 a) de la práctica 1) (base de datos de facturas/clientes), cree un programa java que implemente un Alta y listado de clientes. El programa puede ser hecho sin Interfaz gráfica, sólo en línea de comandos.

Nota: Resolver utilizando los motores que se utilizaron en la creación de las tablas.

Idea

Es el uso más típico de JDBC: `PreparedStatement` con placeholders `?` para escribir (`INSERT`) y `ResultSet` para leer (`SELECT`). El esqueleto del programa separa *value object* (`Cliente`), *DAO* (`ClienteDao`), *config* (`DbConfig`) y *main* (`ClientesCliApp`); esto permite cambiar entre MySQL y PostgreSQL apenas cambiando el archivo `.properties`.

Solución

Estructura de paquetes (`ar.edu.unrc.bd2.practico4.ej1`):

- `Cliente`: value object inmutable con los cinco campos.
- `ClienteDao`: encapsula JDBC. Recibe la `Connection` desde afuera y expone `insertar` y `listar`.
- `DbConfig`: lee el `.properties`, abre la conexión y expone el nombre completo de la tabla.

- ClientesCliApp: el main con el menú de consola.

Configuración por motor (.properties):

```
# config/db.postgres.properties
db.driver=org.postgresql.Driver
db.url=jdbc:postgresql://localhost:5432/postgres
db.user=postgres
db.password=root
db.table.cliente=practico1a.cliente
```

```
# config/db.mysql.properties
db.driver=com.mysql.cj.jdbc.Driver
db.url=jdbc:mysql://localhost:3306/practico1a_mysql?useSSL=false&serverTimezone=UTC
db.user=root
db.password=root
db.table.cliente=cliente
```

Apertura de conexión:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public Connection createConnection() throws SQLException, ClassNotFoundException {
    Class.forName(driverClass);
    return DriverManager.getConnection(url, user, password);
}
```

Acceso a datos: ClienteDao:

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public int insertar(Connection connection, Cliente cliente) throws SQLException {
    String sql = "INSERT INTO " + clienteTable
        + " (nro_cliente, apellido, nombre, direccion, telefono) VALUES (?, ?, ?, ?, ?)";

    try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
        preparedStatement.setInt(1, cliente.getNroCliente());
        preparedStatement.setString(2, cliente.getApellido());
        preparedStatement.setString(3, cliente.getNombre());
        preparedStatement.setString(4, cliente.getDireccion());
        preparedStatement.setString(5, cliente.getTelefono());
        return preparedStatement.executeUpdate();
    }
}

public List<Cliente> listar(Connection connection) throws SQLException {
    String sql = "SELECT nro_cliente, apellido, nombre, direccion, telefono FROM "
        + clienteTable + " ORDER BY nro_cliente";

    List<Cliente> clientes = new ArrayList<>();

    try (PreparedStatement preparedStatement = connection.prepareStatement(sql);
        ResultSet resultSet = preparedStatement.executeQuery()) {
```

```

        while (resultSet.next()) {
            clientes.add(new Cliente(
                resultSet.getInt("nro_cliente"),
                resultSet.getString("apellido"),
                resultSet.getString("nombre"),
                resultSet.getString("direccion"),
                resultSet.getString("telefono")
            ));
        }
    }
    return clientes;
}

```

Menú de consola:

```

import java.sql.Connection;
import java.util.Scanner;

private static void ejecutarMenu(Scanner scanner, Connection connection,
                                ClienteDao clienteDao) {

    boolean salir = false;
    while (!salir) {
        mostrarMenu();
        String opcion = scanner.nextLine().trim();
        switch (opcion) {
            case "1": altaCliente(scanner, connection, clienteDao); break;
            case "2": listarClientes(connection, clienteDao); break;
            case "0": salir = true; break;
            default: System.out.println("Opcion invalida.");
        }
    }
}

```

Por qué funciona

- `Class.forName(driverClass)` fuerza la carga de la clase del driver. Desde JDBC 4.x los drivers se autorregistran por `ServiceLoader`, pero llamarlo explícitamente es la forma robusta que muestra el teórico y evita problemas si `META-INF/services` no está empaquetado.
- `DriverManager.getConnection(url, user, password)` devuelve la `Connection`. La URL define el motor (`jdbc:postgresql` vs `jdbc:mysql`); el resto del código no cambia.
- `PreparedStatement` con `?` y `setInt/setString` envía sentencia y valores por separado: protege contra *SQL Injection* y permite que el motor cachee el plan.
- `executeUpdate` devuelve cuántas filas modificó (se espera 1 para un alta); `executeQuery` devuelve un `ResultSet`, recorrido con `while (rs.next())`.
- Las llamadas a `rs.getInt("nro_cliente")` usan *nombre de columna* en vez de índice numérico: menos eficiente pero más legible y robusto frente a cambios en el orden del `SELECT`.
- Todo va dentro de `try-with-resources` para que `Connection`, `PreparedStatement` y `ResultSet` se cierren aunque salte una excepción.

En MySQL 8+

La consigna ya pide ambos motores (MySQL y PostgreSQL), y el `.properties` de MySQL aparece en la *Solución*. Los puntos a recordar específicos de MySQL 8+:

- **Driver:** `com.mysql.cj.jdbc.Driver` del artefacto Maven `mysql:mysql-connector-j:8.x` (o el JAR equivalente bajado a `lib/`).
- **URL:** `jdbc:mysql://host:3306/baseDatos?useSSL=false&serverTimezone=UTC`. Sin `serverTimezone` MySQL 8 puede tirar `The server time zone value is unrecognized` al abrir la conexión.
- **Base vs esquema:** en MySQL la “base” es lo que JDBC llama `catalog`; no hay un esquema separado. Por eso el `.properties` apunta directamente a `practico1a_mysql` y la columna `db.table.cliente` no necesita prefijo.
- **Identificadores:** si una columna o tabla choca con una palabra reservada de MySQL, va entre backticks ‘...’ en vez de comillas dobles (PostgreSQL/Oracle usan “...”).

Prueba

```
# Desde practico-04/ejercicio-01
mkdir -p out
javac -d out src/main/java/ar/edu/unrc/bd2/practico4/ej1/*.java

# PostgreSQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej1.ClientesCliApp \
    config/db.postgres.properties

# MySQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej1.ClientesCliApp \
    config/db.mysql.properties
```

3. Ejercicio 2: Invocar la función de máxima cotización

Consigna

Realizar un programa Java que invoque la función implementada en el ejercicio 2 de la práctica 3 (máxima cotización) y muestre los parámetros de salida.

Idea

Es el caso clásico de `CallableStatement`: se invoca una función almacenada del servidor con un parámetro IN (`cod_banco`) y un OUT (`cotizacion_maxima`). En PostgreSQL la función se llama con el `escape call { ? = call func(?) }`; se registra el OUT con `registerOutParameter(1, Types.NUMERIC)` y se lee con `getBigDecimal(1)`.

Solución

Estructura del proyecto:

- `CotizacionMaximaApp`: el `main`; lee el `cod_banco`, invoca el `service`, imprime el resultado y consulta la fila actualizada.
- `CotizacionMaximaService`: encapsula la invocación JDBC con dos caminos: `CallableStatement` (preferido) y `SELECT * FROM funcion(?)` como *fallback*.
- `DbConfig`: lee `driver`, `url`, `credenciales` y las dos consultas (`call` y `select`).

Configuración PostgreSQL:

```
# config/db.postgres.properties
db.driver=org.postgresql.Driver
```

```

db.url=jdbc:postgresql://127.0.0.1:5432/postgres
db.user=postgres
db.password=

# Invocacion por CallableStatement (parametro OUT en posicion 1).
db.function.call={ ? = call practico3_ej2.fn_actualizar_cotizacion_maxima(?) }

# Fallback por SELECT.
db.function.select=SELECT * FROM practico3_ej2.fn_actualizar_cotizacion_maxima(?)

# Verificacion posterior de la fila actualizada.
db.query.banco=SELECT cod_banco, nombre, cotizacion_maxima FROM practico3_ej2.banco WHERE
    cod_banco = ?

```

Invocación con CallableStatement:

```

import java.math.BigDecimal;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Types;

private BigDecimal invocarConCallableStatement(int codBanco, Connection connection)
    throws SQLException {
    try (CallableStatement callableStatement =
        connection.prepareCall(dbConfig.getFunctionCall())) {
        callableStatement.registerOutParameter(1, Types.NUMERIC);
        callableStatement.setInt(2, codBanco);
        callableStatement.execute();
        return callableStatement.getBigDecimal(1);
    }
}

```

Fallback con SELECT:

```

import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

private BigDecimal invocarConSelect(int codBanco, Connection connection)
    throws SQLException {
    try (PreparedStatement preparedStatement =
        connection.prepareStatement(dbConfig.getFunctionSelect())) {
        preparedStatement.setInt(1, codBanco);
        try (ResultSet resultSet = preparedStatement.executeQuery()) {
            if (!resultSet.next()) {
                throw new SQLException("La funcion no retorno filas de salida.");
            }
            return resultSet.getBigDecimal(1);
        }
    }
}

public BigDecimal invocarFuncion(int codBanco, Connection connection) throws SQLException {
    try {
        return invocarConCallableStatement(codBanco, connection);
    } catch (SQLException e) {
        return invocarConSelect(codBanco, connection);
    }
}

```

Verificación posterior:

```
public BancoCotizacion consultarBanco(int codBanco, Connection connection) throws
SQLException {
    try (PreparedStatement preparedStatement =
        connection.prepareStatement(dbConfig.getBancoQuery())) {
        preparedStatement.setInt(1, codBanco);
        try (ResultSet resultSet = preparedStatement.executeQuery()) {
            if (!resultSet.next()) return null;
            return new BancoCotizacion(
                resultSet.getInt("cod_banco"),
                resultSet.getString("nombre"),
                resultSet.getBigDecimal("cotizacion_maxima")
            );
        }
    }
}
```

Por qué funciona

- `prepareCall` es el método de `Connection` para crear un `CallableStatement`. La sintaxis estándar JDBC para llamar a una función con OUT es el *escape call* { ? = call func(?) }, que el driver traduce al SQL nativo del motor.
- `registerOutParameter(1, Types.NUMERIC)` le dice al driver que el primer placeholder es de salida y de tipo NUMERIC (el tipo SQL de `cotizacion_maxima`). `Types` viene de `java.sql.Types`.
- `getBigDecimal(1)` lee el valor de salida en el placeholder 1 como `BigDecimal`, que es la correspondencia natural de NUMERIC en Java según la tabla de tipos JDBC.
- El *fallback* con `SELECT * FROM funcion(?)` existe porque algunos drivers/versiones de PostgreSQL no aceptan el *escape call* para funciones escalares. `invocarFuncion` prueba primero el camino limpio y, ante una `SQLException`, recurre al `SELECT`.

En MySQL 8+

La función de PostgreSQL del Práctico 3 (con parámetros IN/OUT) se traduce en MySQL a un PROCEDURE con IN `p_cod_banco`, OUT `p_cotizacion_maxima` (porque MySQL no soporta OUT en FUNCTION; solo en PROCEDURE). El *escape call* se invoca con { CALL `sp(?, ?)` }, registrando el OUT en la posición 2:

```
# config/db.mysql.properties
db.driver=com.mysql.cj.jdbc.Driver
db.url=jdbc:mysql://localhost:3306/practico3_ej2_mysql?useSSL=false&serverTimezone=UTC
db.user=root
db.password=root

# IN en posicion 1, OUT en posicion 2 (orden de la declaracion del PROCEDURE).
db.function.call={ CALL sp_actualizar_cotizacion_maxima(?, ?) }

# Verificacion
db.query.banco=SELECT cod_banco, nombre, cotizacion_maxima FROM banco WHERE cod_banco = ?
```

```
import java.math.BigDecimal;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Types;

private BigDecimal invocarConCallableStatementMySQL(int codBanco, Connection connection)
```

```

        throws SQLException {
    try (CallableStatement callableStatement =
        connection.prepareCall("{ CALL sp_actualizar_cotizacion_maxima(?, ?) }")) {
        // IN: cod_banco
        callableStatement.setInt(1, codBanco);
        // OUT: cotizacion_maxima (DECIMAL en MySQL)
        callableStatement.registerOutParameter(2, Types.DECIMAL);
        callableStatement.execute();
        return callableStatement.getBigDecimal(2);
    }
}

```

Qué cambia respecto de PostgreSQL:

- FUNCTION con OUT se transforma en PROCEDURE: el escape call cambia de { ? = call f(?) } (con OUT en posición 1, IN en 2) a { CALL p(?, ?) } (IN en 1, OUT en 2, según el orden de declaración del procedimiento).
- El tipo registrado es Types.DECIMAL (en MySQL la columna se declaró DECIMAL(12,4)); en PostgreSQL es Types.NUMERIC. Ambos se leen con getBigDecimal.
- El *fallback* con SELECT * FROM funcion(?) *no aplica* en MySQL: no hay funciones tabulares; si el CALL no es soportado, no hay un atajo equivalente.

Prueba

Salida esperada (PostgreSQL):

```

Funcion invocada: practico3_ej2.fn_actualizar_cotizacion_maxima
Parametro IN  p_cod_banco      = 1
Parametro OUT p_cotizacion_maxima = 1245.7500
Banco actualizado -> cod_banco=1, nombre=Banco Rio, cotizacion_maxima=1245.7500

```

Compilación y ejecución:

```

# Desde practico-04/ejercicio-02
mkdir -p out
javac -d out src/main/java/ar/edu/unrc/bd2/practico4/ej2/*.java

# Interactivo (pide cod_banco por consola)
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej2.CotizacionMaximaApp \
    config/db.postgres.properties

# O con cod_banco como argumento
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej2.CotizacionMaximaApp \
    config/db.postgres.properties 1

# Variante MySQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej2.CotizacionMaximaApp \
    config/db.mysql.properties 1

```

4. Ejercicio 3: Listado de metadatos (PK, UQ, FK)

Consigna

Realizar un programa Java que liste las tablas de la base de datos del ejercicio 1 c) de la

práctica 1) de repaso de SQL (bases de datos cliente, automóviles, etc), debe listar las tablas, sus columnas, clave primaria, claves únicas y foráneas.

Nota: Resolver utilizando los motores que se utilizaron en la creación de las bases de datos.

Idea

El programa no se ocupa de datos, sino de *metadatos*: la estructura de las tablas. JDBC expone esta información en `DatabaseMetaData`, una interfaz que estandariza los `ResultSet` con columnas como `TABLE_NAME`, `COLUMN_NAME`, `PK_NAME`, `FKCOLUMN_NAME`, etc. Eso permite escribir un programa portable entre motores: lo único que cambia entre PostgreSQL, Oracle y MySQL es el filtro de `catalog/schemaPattern`, que viaja en el `.properties`.

Solución

Estructura:

- `MetadataExplorerApp`: main mínimo; abre conexión, obtiene `DatabaseMetaData`, delega en el reporter.
- `MetadataReporter`: cinco pasos: cargar tablas, columnas, PK, UQ, FK.
- `DbConfig`: filtros para `getTables` (`catalog`, `schemaPattern`, `tableNamePattern`, `tableTypes`).

Configuración PostgreSQL:

```
# config/db.postgres.properties
db.driver=org.postgresql.Driver
db.url=jdbc:postgresql://127.0.0.1:5432/postgres
db.user=postgres
db.password=

db.catalog=
db.schemaPattern=practico1c
db.tableNamePattern=%
db.table.types=TABLE
```

Configuración Oracle:

```
# config/db.oracle.properties
db.driver=oracle.jdbc.OracleDriver
db.url=jdbc:oracle:thin:@//localhost:1521/XE
db.user=PRACTIC01C
db.password=PRACTIC01C

# En Oracle, el "schema" es el usuario; debe ir en MAYUSCULAS.
db.catalog=
db.schemaPattern=PRACTIC01C
db.tableNamePattern=%
db.table.types=TABLE
```

Carga de tablas:

```
import java.sql.DatabaseMetaData;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
```

```

private List<TableRef> loadTables(DatabaseMetaData metaData, DbConfig config)
    throws SQLException {
    List<TableRef> tables = new ArrayList<>();
    try (ResultSet rs = metaData.getTables(
        config.getCatalog(),
        config.getSchemaPattern(),
        config.getTableNamePattern(),
        config.getTableTypes())) {
        while (rs.next()) {
            tables.add(new TableRef(
                rs.getString("TABLE_CAT"),
                rs.getString("TABLE_SCHEM"),
                rs.getString("TABLE_NAME")
            ));
        }
    }
    tables.sort(Comparator
        .comparing((TableRef t) -> nullSafe(t.schema))
        .thenComparing(t -> nullSafe(t.name)));
    return tables;
}

```

Columnas:

```

private void printColumns(DatabaseMetaData metaData, TableRef table) throws SQLException {
    try (ResultSet rs = metaData.getColumns(table.catalog, table.schema, table.name, "%")) {
        while (rs.next()) {
            String columnName = rs.getString("COLUMN_NAME");
            String typeName = rs.getString("TYPE_NAME");
            int dataType = rs.getInt("DATA_TYPE");
            int size = rs.getInt("COLUMN_SIZE");
            int decimalDigits = rs.getInt("DECIMAL_DIGITS");
            String nullable = rs.getString("IS_NULLABLE");

            String typeDescription = buildTypeDescription(typeName, dataType, size,
                decimalDigits);
            String nullDescription = "YES".equalsIgnoreCase(nullable) ? "NULL" : "NOT NULL";

            System.out.println("- " + columnName + " : " + typeDescription
                + " (" + nullDescription + ")");
        }
    }
}

```

Clave primaria (con KEY_SEQ para PK compuestas):

```

import java.util.Map;
import java.util.TreeMap;

private void printPrimaryKey(DatabaseMetaData metaData, TableRef table) throws SQLException {
    Map<Short, String> pkColumns = new TreeMap<>();
    String pkName = null;

    try (ResultSet rs = metaData.getPrimaryKeys(table.catalog, table.schema, table.name)) {
        while (rs.next()) {
            pkName = rs.getString("PK_NAME");
            short keySeq = rs.getShort("KEY_SEQ");
            pkColumns.put(keySeq, rs.getString("COLUMN_NAME"));
        }
    }

    if (pkColumns.isEmpty()) {

```

```

        System.out.println("PK:\n- (sin clave primaria)");
        return;
    }
    System.out.println("PK:\n- " + (pkName != null ? pkName : "(sin nombre)")
        + " " + new ArrayList<>(pkColumns.values()));
}

```

Claves únicas (no hay `getUniqueKeys`: se usa `getIndexInfo` y se filtran las que coinciden con la PK):

```

try (ResultSet rs = metaData.getIndexInfo(table.catalog, table.schema, table.name,
    /* unique = */ true, /* approximate = */ false)) {
    while (rs.next()) {
        short type = rs.getShort("TYPE");
        if (type == DatabaseMetaData.tableIndexStatistic) continue;

        String indexName = rs.getString("INDEX_NAME");
        String columnName = rs.getString("COLUMN_NAME");
        short ordinalPosition = rs.getShort("ORDINAL_POSITION");
        uniqueIndexes.computeIfAbsent(indexName, k -> new TreeMap<>())
            .put(ordinalPosition, columnName);
    }
}
// Despues se descartan los indices unicos cuyas columnas coinciden con la PK.

```

Claves foráneas:

```

try (ResultSet rs = metaData.getImportedKeys(table.catalog, table.schema, table.name)) {
    while (rs.next()) {
        String fkName = rs.getString("FK_NAME");
        short keySeq = rs.getShort("KEY_SEQ");
        FkPart part = new FkPart(
            rs.getString("FKCOLUMN_NAME"),
            rs.getString("PKTABLE_SCHEM"),
            rs.getString("PKTABLE_NAME"),
            rs.getString("PKCOLUMN_NAME")
        );
        groupedFks.computeIfAbsent(fkName, k -> new TreeMap<>()).put(keySeq, part);
    }
}

```

Punto de entrada:

```

import java.io.IOException;
import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.SQLException;

public static void main(String[] args) {
    String configPath = args.length > 0 ? args[0] : "config/db.postgres.properties";
    DbConfig config;
    try {
        config = DbConfig.fromFile(configPath);
    } catch (IOException | IllegalArgumentException e) {
        System.err.println("Error cargando config: " + e.getMessage());
        System.exit(1); return;
    }

    try (Connection connection = config.openConnection()) {
        DatabaseMetaData metaData = connection.getMetaData();
        System.out.println("Conexion OK -> "

```

```

        + metaData.getDatabaseProductName() + " "
        + metaData.getDatabaseProductVersion());

    new MetadataReporter().printMetadata(metaData, config);
} catch (ClassNotFoundException e) {
    System.err.println("Driver no encontrado: " + e.getMessage());
} catch (SQLException e) {
    System.err.println("Error SQL: " + e.getMessage());
}
}

```

Por qué funciona

- `getTables(catalog, schemaPattern, tableNamePattern, types)` devuelve un `ResultSet` con las tablas filtradas. Las columnas (`TABLE_CAT`, `TABLE_SCHEM`, `TABLE_NAME`, `TABLE_TYPE`) están definidas por el estándar JDBC, así que el código no depende del motor.
- `getColumns` devuelve cada columna con `DATA_TYPE` (código de `java.sql.Types`), `TYPE_NAME` (nombre nativo, ej. `VARCHAR2`, `NUMBER`), `COLUMN_SIZE`, `DECIMAL_DIGITS` y `IS_NULLABLE` ("YES"/"NO").
- `getPrimaryKeys` puede devolver varias filas si la PK es compuesta. `KEY_SEQ` indica el orden dentro de la PK; por eso se acumula en un `TreeMap<Short, String>` para ordenar automáticamente.
- JDBC no tiene `getUniqueKeys`: se piden los índices únicos con `getIndexInfo(..., unique=true, ...)` y se descartan los que coinciden exactamente con la PK (que también genera un índice único).
- `getImportedKeys` devuelve las FK *salientes* de la tabla (las que esta tabla declara hacia otras); `getExportedKeys` sería el camino inverso. Una FK también puede ser compuesta, así que `KEY_SEQ` se usa de nuevo.
- Los filtros varían por motor: en PostgreSQL, `schemaPattern = 'practico1c'`; en Oracle no hay esquema separado del usuario, así que va `'PRACTIC01C'` en mayúsculas; en MySQL no hay esquema sino *catalog*, así que `catalog = 'practico1c_mysql'` y `schemaPattern = null`.

En MySQL 8+

El código Java de `DatabaseMetaData` es exactamente el mismo: lo único que cambia es el `.properties`. La diferencia conceptual a tener clara es que MySQL usa el slot `catalog` de JDBC para el nombre de la base, y `schemaPattern` queda en blanco (o `null`):

```

# config/db.mysql.properties
db.driver=com.mysql.cj.jdbc.Driver
db.url=jdbc:mysql://localhost:3306/practico1c_mysql?useSSL=false&serverTimezone=UTC&nullCatalogMeansCurrent=false
db.user=root
db.password=root

# En MySQL, el "catalog" es la base; el "schema" no aplica.
db.catalog=practico1c_mysql
db.schemaPattern=
db.tableNamePattern=%
db.table.types=TABLE

```

Particularidades de MySQL relevantes para metadatos:

- `nullCatalogMeansCurrent=false` en la URL evita el comportamiento heredado donde

catalog=null matcheaba la base actual: con `false`, MySQL respeta el filtro tal como lo manda el cliente.

- En `getColumns`, el `TYPE_NAME` aparece como `INT`, `VARCHAR`, `DECIMAL`, `DATETIME`, etc.; nada de `VARCHAR2` ni `NUMBER` como en Oracle, ni `INT4` como devuelve PostgreSQL.
- Para identificadores con caracteres reservados, MySQL usa backticks ``...``; PostgreSQL y Oracle usan comillas dobles `"..."`. El programa no concatena identificadores “a mano”, así que esto no afecta al código del Ejercicio 3.
- Para columnas `AUTO_INCREMENT`, MySQL las marca con `IS_AUTOINCREMENT = YES` en el `ResultSet` de `getColumns` (PostgreSQL hace lo mismo con `IDENTITY`; Oracle 10/11 no, porque no tiene `IDENTITY` hasta 12c).

Prueba

```
# Desde practico-04/ejercicio-03
mkdir -p out
javac -d out src/main/java/ar/edu/unrc/bd2/practico4/ej3/*.java

# PostgreSQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej3.MetadataExplorerApp \
    config/db.postgres.properties

# Oracle
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej3.MetadataExplorerApp \
    config/db.oracle.properties

# MySQL
java -cp "out:lib/*" \
    ar.edu.unrc.bd2.practico4.ej3.MetadataExplorerApp \
    config/db.mysql.properties
```

Salida esperada (PostgreSQL):

```
Conexion OK -> PostgreSQL 14.10
=====
Tabla: practico1c.cliente
=====
Columnas:
- dni : INT4 (NOT NULL)
- nombre : VARCHAR(60) (NOT NULL)
- apellido : VARCHAR(60) (NOT NULL)
- direccion : VARCHAR(120) (NULL)
- tarifa : INT4 (NOT NULL)
PK:
- pk_cliente [dni]
Claves unicas:
- (sin claves unicas adicionales)
Claves foraneas:
- (sin claves foraneas)

=====
Tabla: practico1c.accidente
=====
...
Claves foraneas:
- fk_acc_cliente [dni] -> practico1c.cliente [dni]
- fk_acc_taller [nro_taller] -> practico1c.taller [nro_taller]
- fk_acc_automovil [patente] -> practico1c.automovil [patente]
```

5. Decisiones transversales

- **Sin frameworks ORM.** Se usa únicamente el paquete `java.sql` (`Connection`, `PreparedStatement`, `CallableStatement`, `ResultSet`, `DatabaseMetaData`). El enunciado pide JDBC: nada de Hibernate, JPA, Spring Data.
- **Recursos cerrados con `try-with-resources`.** El compilador genera el `finally` con `close()`, incluso si el cuerpo lanza excepción. Es el patrón seguro desde Java 7.
- **Configuración externa.** `driver`, `url`, `user`, `password` y los nombres de tabla/función viven en `.properties`. El binario es independiente del entorno.
- **Independencia de motor donde tiene sentido.** El Ejercicio 1 cambia entre MySQL y PostgreSQL apenas cambiando el `.properties`; el Ejercicio 3 lo hace entre PostgreSQL, Oracle y MySQL.
- **Sentencias parametrizadas.** Todos los `INSERT/SELECT/CALL` usan `?` y `PreparedStatement/CallableStatement`. Cero concatenación, cero ventana para SQL Injection.
- **Errores diferenciados.** Cada `main` distingue `ClassNotFoundException` (driver ausente), `IOException` (problema con el `.properties`) y `SQLException` (problema con la base o la sentencia).

Conclusión

Los tres ejercicios cubren los tres patrones canónicos de uso de JDBC: *ABM con `PreparedStatement`*, *invocación de stored procedure con `CallableStatement`* y *exploración de metadatos con `DatabaseMetaData`*. La estructura repetida (`App` + `Service/Dao` + `DbConfig` + `.properties`) deja claro cómo cambia la API según el caso de uso, y los snippets “En MySQL 8+” agregan la portabilidad mínima necesaria para correr cada ejercicio también sobre MySQL: cambiar el driver, el formato de URL (`useSSL`, `serverTimezone`), y entender que MySQL usa *catalog* donde PostgreSQL y Oracle usan *schema*.